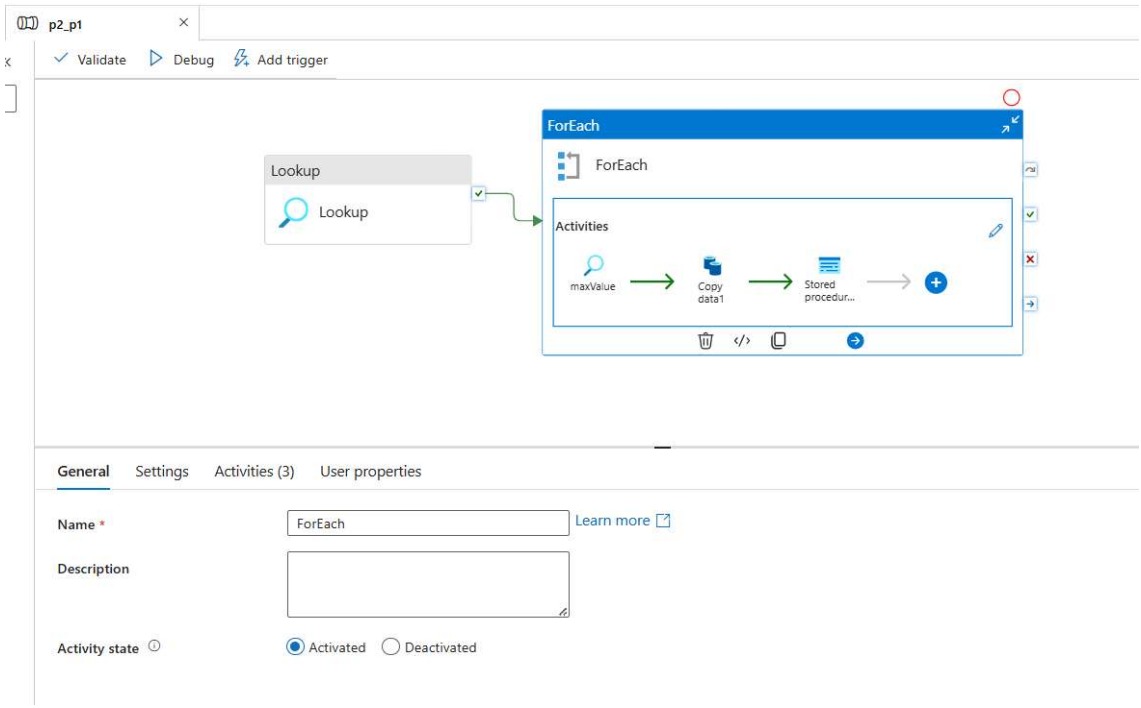
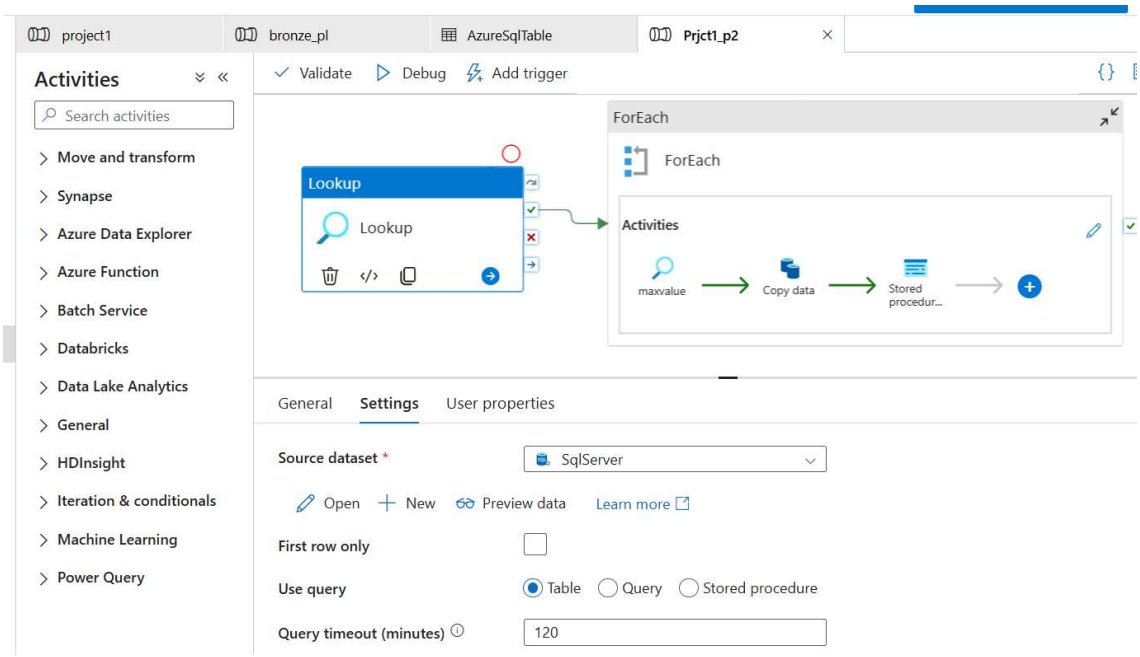
Project 2
ADF and Databricks:

Bronze Layer:

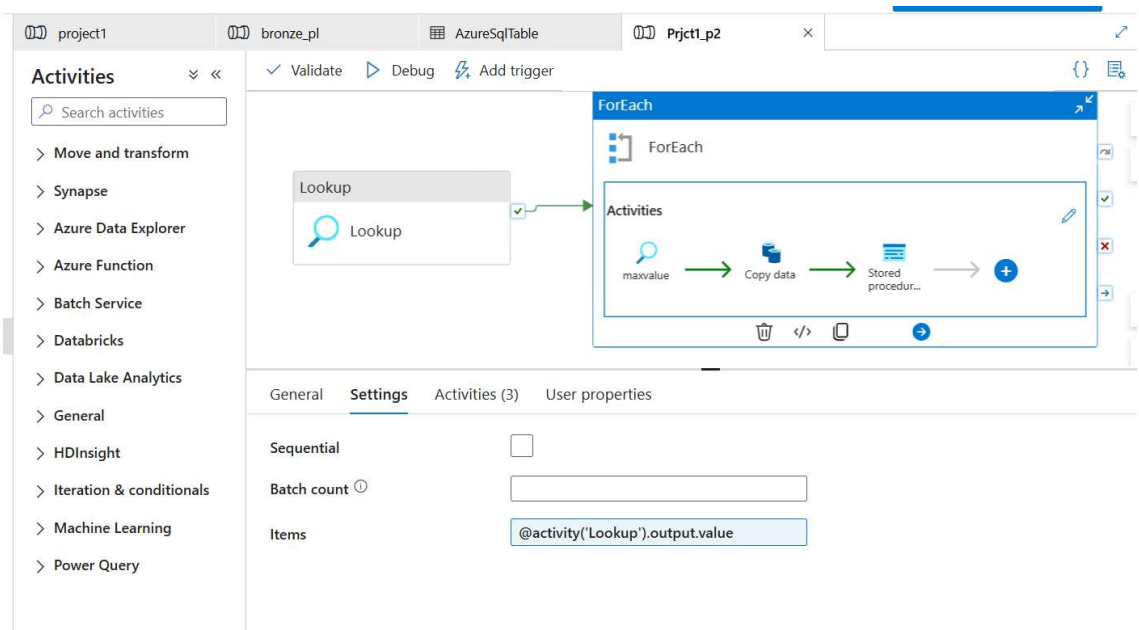
Execute pipeline>On prem to ADLSG2



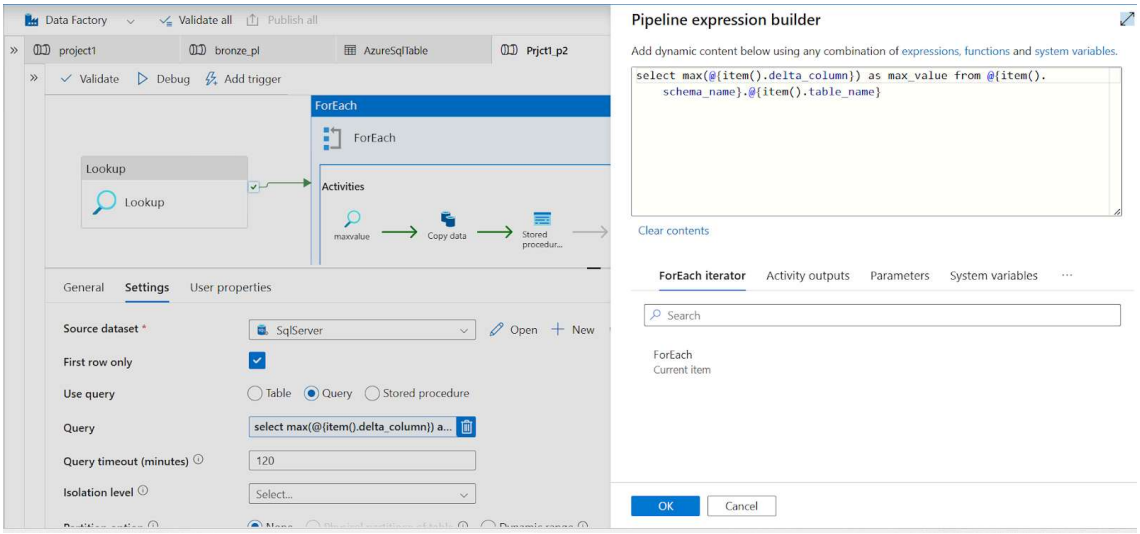
Looking up the data using look up activity.



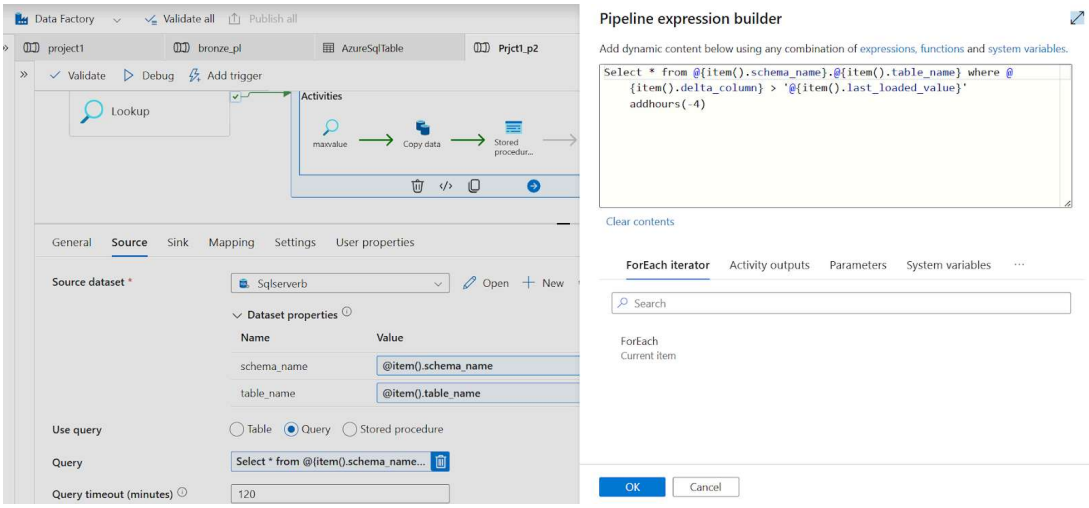
Passing its output value to foreach activity.



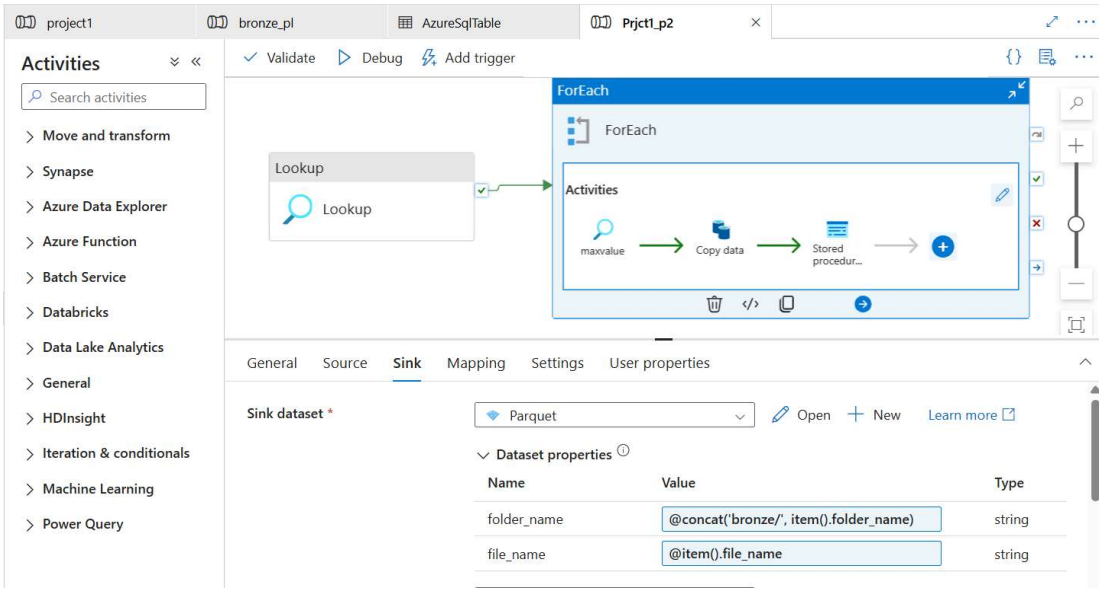
Adding a lookup activity to select max value from the delta column.



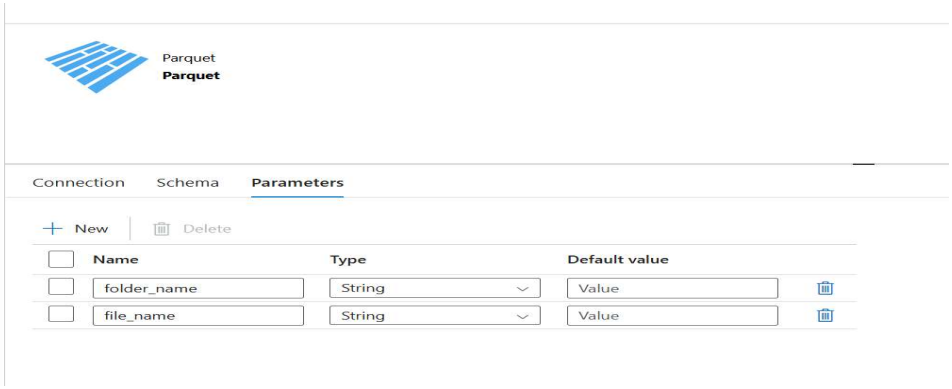
On success of max value look up, copy data. Source is on prem sql data, adding a select query where the delta column is greater than last loaded value. Adding negative hours for dealing with late arrival data.



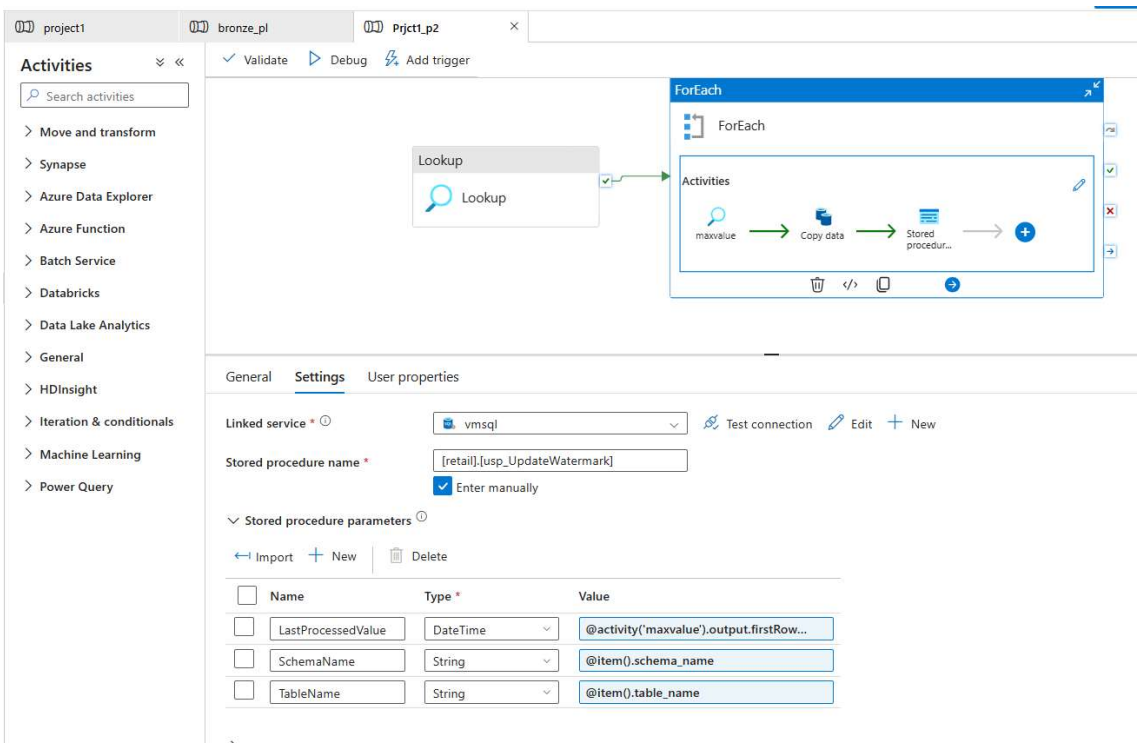
Sinking to the desired location i.e., bronze folder in parquet format.



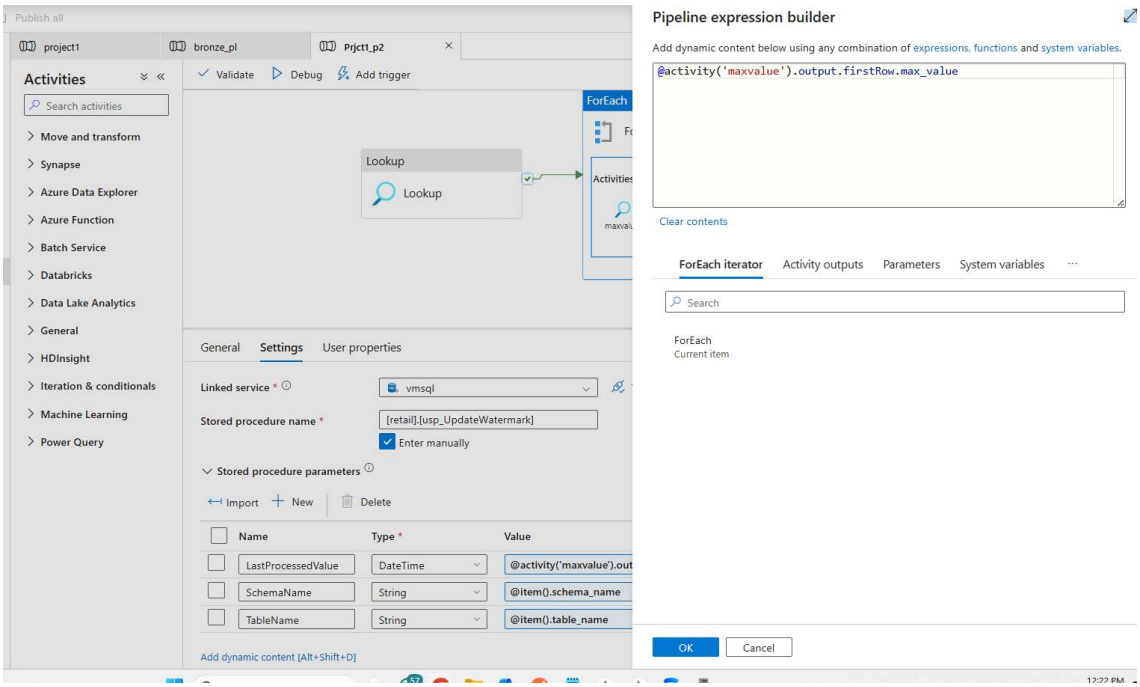
Adding parameters.



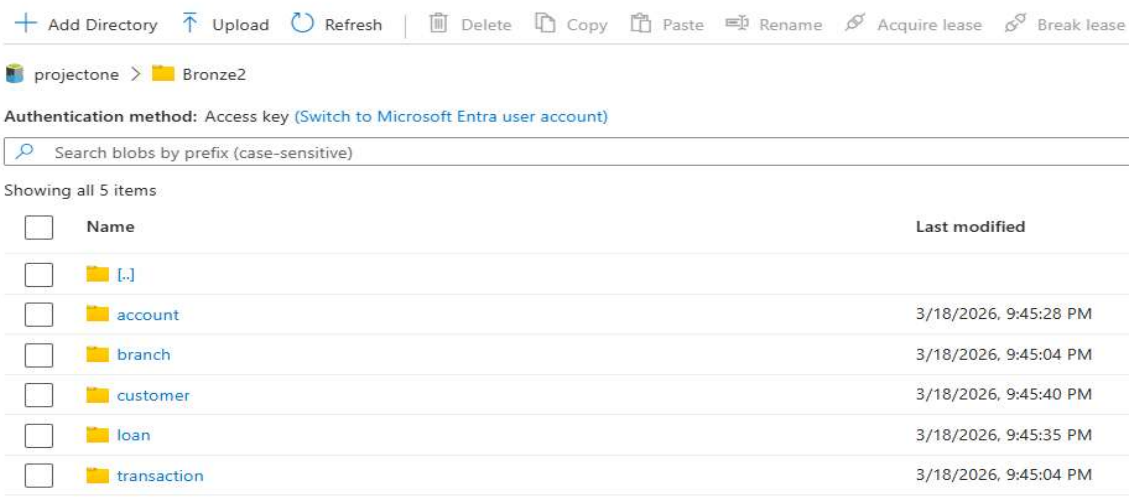
Stored procedure for dynamic autoloading:



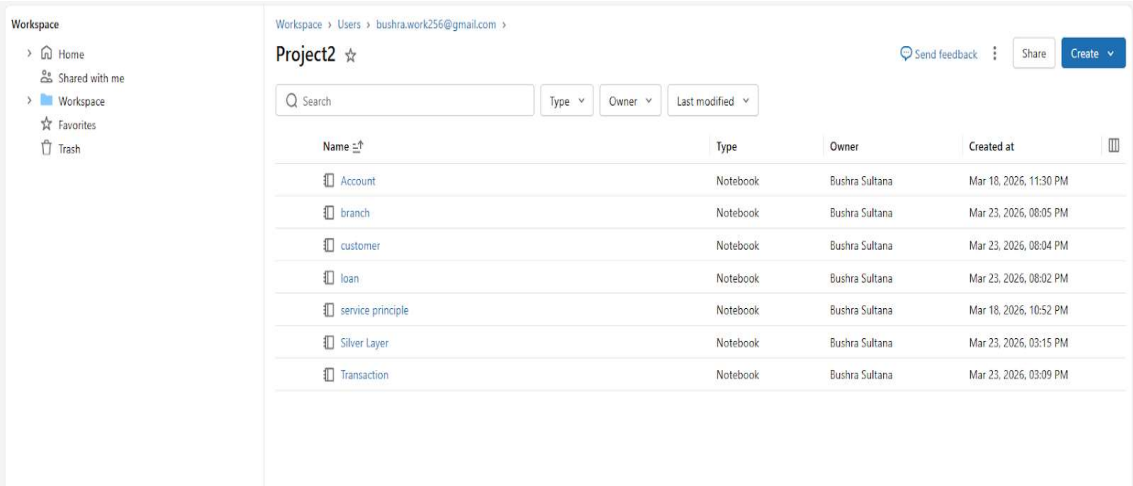
Expression for last processed value:



Result of the pipeline debug:



Silver Layer:



Establishing connection between cloud storage account and data bricks using service principal.

To fetch this connection we use

%run=link
Path of the other notebook
(%run file pathname)

i.e., the path of the service principal notebook

Yesterday (1s)

```python
# Databricks notebook source
dbutils.secrets.listScopes()
```

```
[SecretScope(name='dp')]
```

Yesterday (<1s)

```python
# COMMAND
dbutils.secrets.list('dp')
```

```
[SecretMetadata(key='cc'),
 SecretMetadata(key='clientid'),
 SecretMetadata(key='password'),
 SecretMetadata(key='tenantid')]
```

Yesterday (1s)

```python
# COMMAND ----------

clientid = dbutils.secrets.get("dp", "clientid")
clientsecret = dbutils.secrets.get("dp", "cc")
tenantid = dbutils.secrets.get("dp", "tenantid")
```

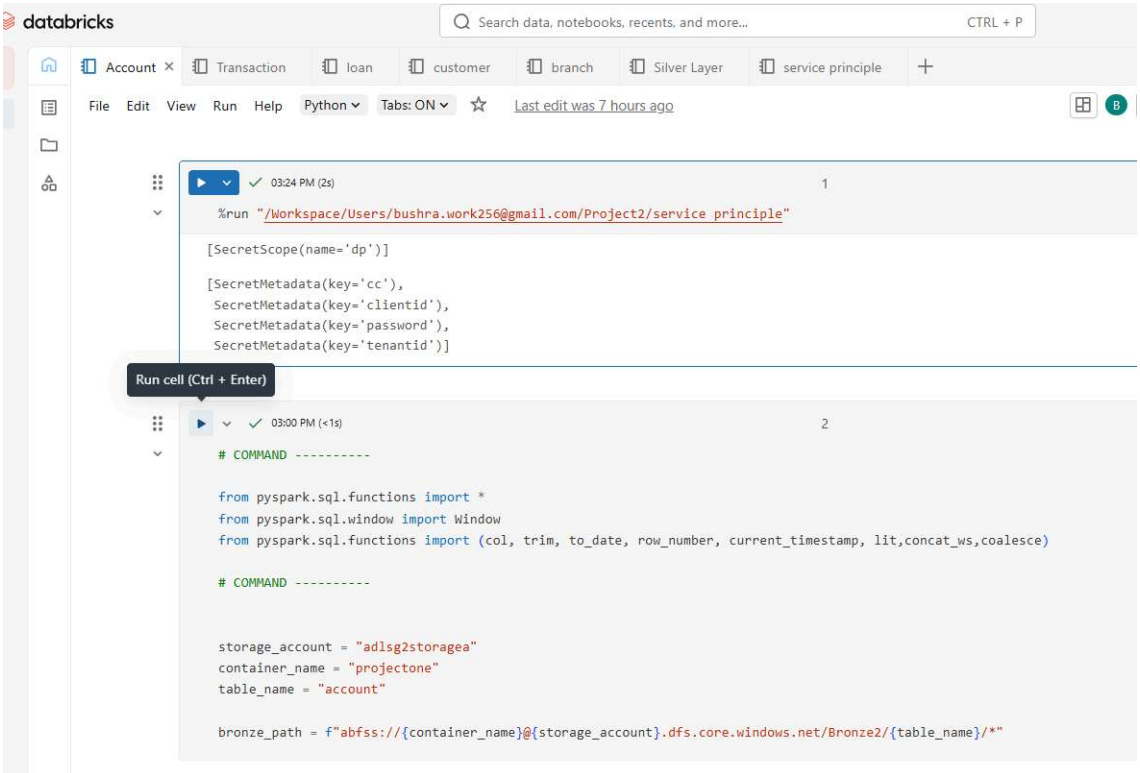
Yesterday (1s) 4

```python
# COMMAND ----------

storage_account = "adlsg2storagea"

spark.conf.set(
    f"fs.azure.account.auth.type.{storage_account}.dfs.core.windows.net",
    "OAuth"
)

spark.conf.set(
    f"fs.azure.account.oauth.provider.type.{storage_account}.dfs.core.windows.net",
    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider"
)

spark.conf.set(
    f"fs.azure.account.oauth2.client.id.{storage_account}.dfs.core.windows.net",
    dbutils.secrets.get("dp", "clientid")
)

spark.conf.set(
    f"fs.azure.account.oauth2.client.secret.{storage_account}.dfs.core.windows.net",
    dbutils.secrets.get("dp", "cc")
)

spark.conf.set(
    f"fs.azure.account.oauth2.client.endpoint.{storage_account}.dfs.core.windows.net",
    f"https://login.microsoftonline.com/{dbutils.secrets.get('dp', 'tenantid')}/oauth2/token"
)
```

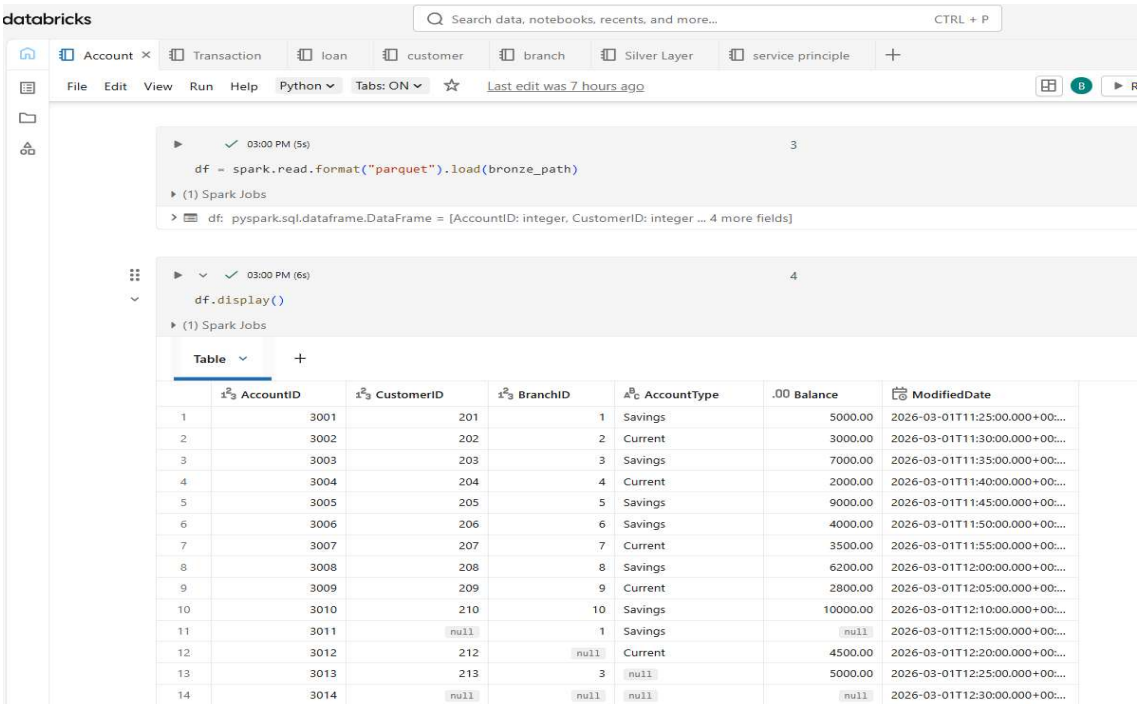
Creating five notebooks one for each table and performing cleaning and null value removal.

Account:

Importing the path of the service principal notebook to utilise the connection.

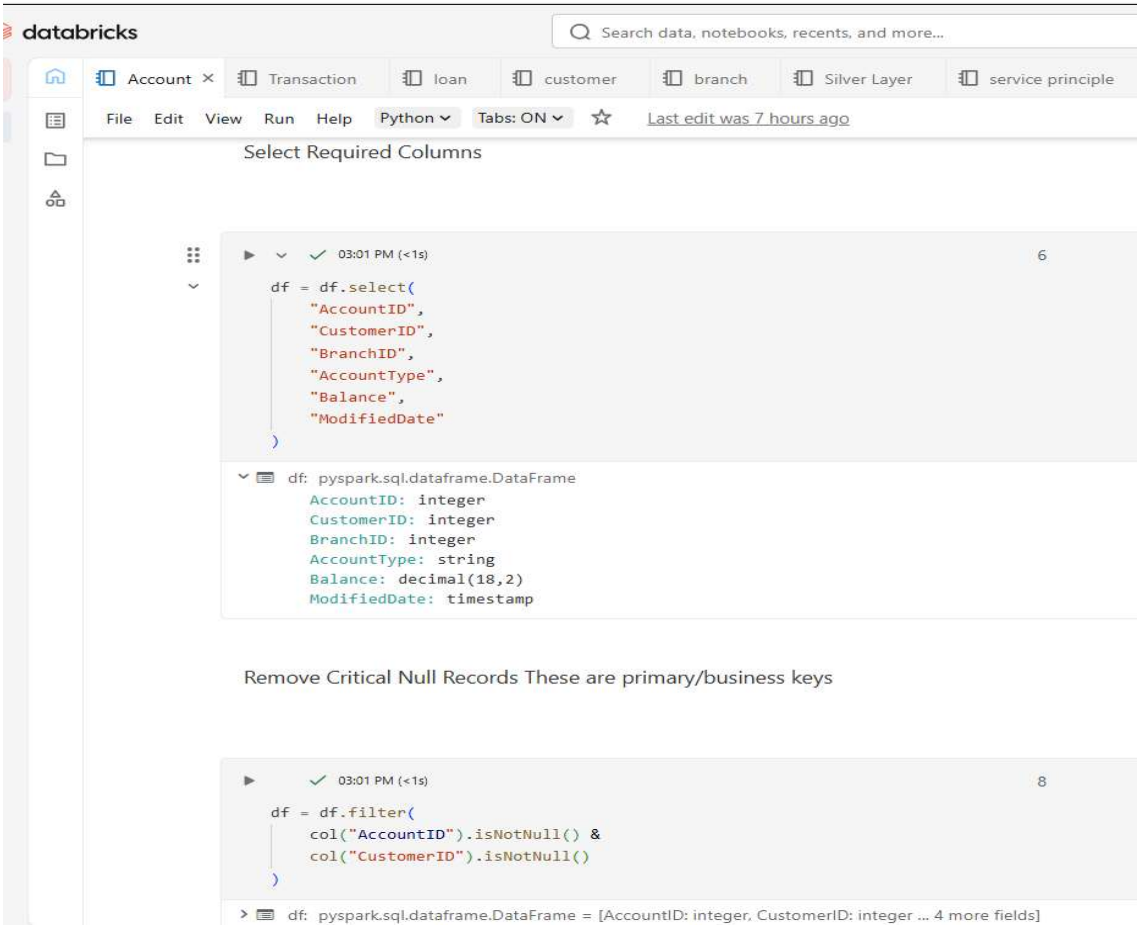


Fetching the table from the bronze:

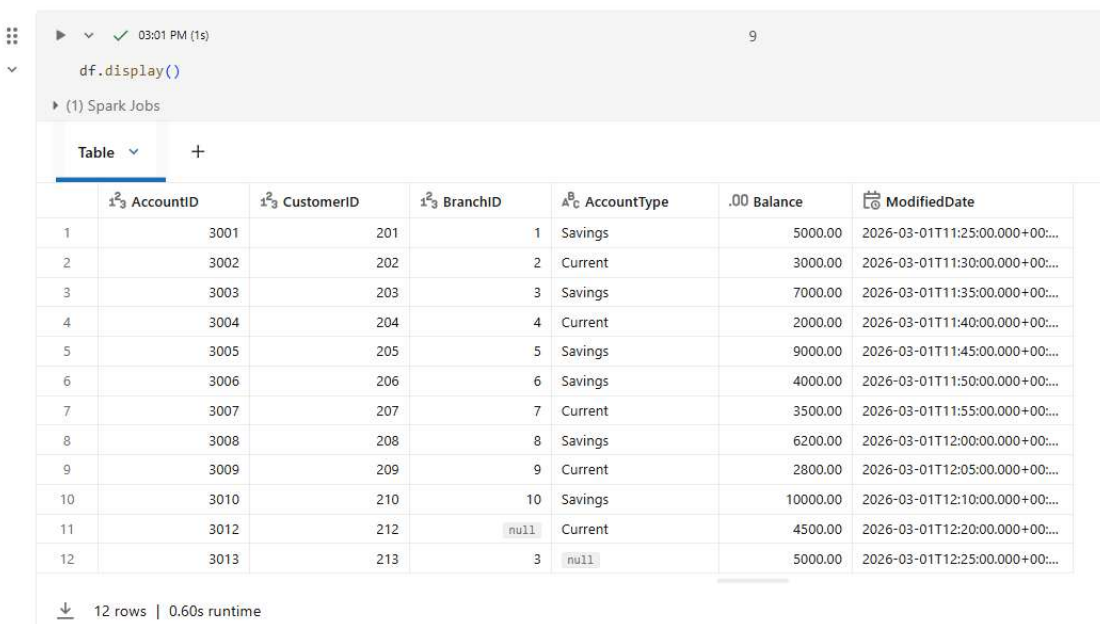


	AccountID	CustomerID	BranchID	AccountType	Balance	ModifiedDate
1	3001	201	1	Savings	5000.00	2026-03-01T11:25:00.000+00:...
2	3002	202	2	Current	3000.00	2026-03-01T11:30:00.000+00:...
3	3003	203	3	Savings	7000.00	2026-03-01T11:35:00.000+00:...
4	3004	204	4	Current	2000.00	2026-03-01T11:40:00.000+00:...
5	3005	205	5	Savings	9000.00	2026-03-01T11:45:00.000+00:...
6	3006	206	6	Savings	4000.00	2026-03-01T11:50:00.000+00:...
7	3007	207	7	Current	3500.00	2026-03-01T11:55:00.000+00:...
8	3008	208	8	Savings	6200.00	2026-03-01T12:00:00.000+00:...
9	3009	209	9	Current	2800.00	2026-03-01T12:05:00.000+00:...
10	3010	210	10	Savings	10000.00	2026-03-01T12:10:00.000+00:...
11	3011	null	1	Savings	null	2026-03-01T12:15:00.000+00:...
12	3012	212	null	Current	4500.00	2026-03-01T12:20:00.000+00:...
13	3013	213	3	null	5000.00	2026-03-01T12:25:00.000+00:...
14	3014	null	null	null	null	2026-03-01T12:30:00.000+00:...

Selecting few or all columns of the table and filtering the null values over the columns:



Display:



	AccountID	CustomerID	BranchID	AccountType	Balance	ModifiedDate
1	3001	201	1	Savings	5000.00	2026-03-01T11:25:00.000+00:...
2	3002	202	2	Current	3000.00	2026-03-01T11:30:00.000+00:...
3	3003	203	3	Savings	7000.00	2026-03-01T11:35:00.000+00:...
4	3004	204	4	Current	2000.00	2026-03-01T11:40:00.000+00:...
5	3005	205	5	Savings	9000.00	2026-03-01T11:45:00.000+00:...
6	3006	206	6	Savings	4000.00	2026-03-01T11:50:00.000+00:...
7	3007	207	7	Current	3500.00	2026-03-01T11:55:00.000+00:...
8	3008	208	8	Savings	6200.00	2026-03-01T12:00:00.000+00:...
9	3009	209	9	Current	2800.00	2026-03-01T12:05:00.000+00:...
10	3010	210	10	Savings	10000.00	2026-03-01T12:10:00.000+00:...
11	3012	212	null	Current	4500.00	2026-03-01T12:20:00.000+00:...
12	3013	213	3	null	5000.00	2026-03-01T12:25:00.000+00:...

12 rows | 0.60s runtime

Replacing nulls using fillna function:

Handle Remaining Nulls

03:01 PM (<1s) 11

```
df = df.fillna({
    "BranchID": -1,                # unknown branch
    "AccountType": "UNKNOWN",
    "Balance": 0
})
```

df: pyspark.sql.dataframe.DataFrame = [AccountID: integer, CustomerID: integer ... 4 more fields]

03:03 PM (1s) 12

```
df.display()
```

(1) Spark Jobs

Table +

	AccountID	CustomerID	BranchID	AccountType	Balance	ModifiedDate
1	3001	201	1	Savings	5000.00	2026-03-01T11:25:00.000+00:...
2	3002	202	2	Current	3000.00	2026-03-01T11:30:00.000+00:...
3	3003	203	3	Savings	7000.00	2026-03-01T11:35:00.000+00:...
4	3004	204	4	Current	2000.00	2026-03-01T11:40:00.000+00:...
5	3005	205	5	Savings	9000.00	2026-03-01T11:45:00.000+00:...
6	3006	206	6	Savings	4000.00	2026-03-01T11:50:00.000+00:...
7	3007	207	7	Current	3500.00	2026-03-01T11:55:00.000+00:...
8	3008	208	8	Savings	6200.00	2026-03-01T12:00:00.000+00:...
9	3009	209	9	Current	2800.00	2026-03-01T12:05:00.000+00:...
10	3010	210	10	Savings	10000.00	2026-03-01T12:10:00.000+00:...
11	3012	212	-1	Current	4500.00	2026-03-01T12:20:00.000+00:...
12	3013	213	3	UNKNOWN	5000.00	2026-03-01T12:25:00.000+00:...

12 rows | 0.64s runtime

Type casting and trimming:

Fix Data Types

03:03 PM (<1s) 14

```
df = df.withColumn("AccountID", col("AccountID").cast("int")) \
       .withColumn("CustomerID", col("CustomerID").cast("int")) \
       .withColumn("BranchID", col("BranchID").cast("int")) \
       .withColumn("Balance", col("Balance").cast("double")) \
       .withColumn("ModifiedDate", to_timestamp(col("ModifiedDate")))
```

df: pyspark.sql.dataframe.DataFrame = [AccountID: integer, CustomerID: integer ... 4 more fields]

Clean Strings

03:03 PM (<1s) 16

```
from pyspark.sql.functions import upper, trim

df = df.withColumn("AccountType", upper(trim(col("AccountType"))))
```

df: pyspark.sql.dataframe.DataFrame = [AccountID: integer, CustomerID: integer ... 4 more fields]

Remove Duplicates

Yesterday (<1s) 18

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

window_spec = Window.partitionBy("AccountID").orderBy(col("ModifiedDate").desc())

df = df.withColumn("rn", row_number().over(window_spec)) \
       .filter(col("rn") == 1) \
       .drop("rn")
```

Business logic, adding audit column using withColumn function:

Business Rule Validation

Yesterday (<1s)

```
df = df.filter(col("Balance") >= 0)
```

df: pyspark.sql.dataframe.DataFrame = [AccountID: integer, CustomerID: integer ... 4 more fields]

Add Audit Column

Yesterday (<1s)

```
from pyspark.sql.functions import current_timestamp

df = df.withColumn("ingestion_time", current_timestamp())
```

df: pyspark.sql.dataframe.DataFrame = [AccountID: integer, CustomerID: integer ... 5 more fields]

Write to Silver

Yesterday (6s)

```
df.write.format("delta") \
  .mode("overwrite") \
  .saveAsTable("project2.silver.account")
```

(2) Spark Jobs

Yesterday (2s)

```
%sql
select * from project2.silver.account
```

(2) Spark Jobs

_sqldf: pyspark.sql.dataframe.DataFrame = [AccountID: integer, CustomerID: integer ... 5 more fields]

Then creating a delta format table to store clean and filtered tables in the silver layer.

```
.saveAsTable("project2.silver.account")
```

(2) Spark Jobs

Yesterday (2s) 25

```
%sql
select * from project2.silver.account
```

(2) Spark Jobs

_sqldf: pyspark.sql.dataframe.DataFrame = [AccountID: integer, CustomerID: integer ... 5 more fields]

Table +

	AccountID	CustomerID	BranchID	AccountType	Balance	ModifiedDate	ingestion_time
1	3001	201	1	SAVINGS	5000	2026-03-01T11:25:00.000+00:...	2026-03-23T02:00:41.868+00:...
2	3002	202	2	CURRENT	3000	2026-03-01T11:30:00.000+00:...	2026-03-23T02:00:41.868+00:...
3	3003	203	3	SAVINGS	7000	2026-03-01T11:35:00.000+00:...	2026-03-23T02:00:41.868+00:...
4	3004	204	4	CURRENT	2000	2026-03-01T11:40:00.000+00:...	2026-03-23T02:00:41.868+00:...
5	3005	205	5	SAVINGS	9000	2026-03-01T11:45:00.000+00:...	2026-03-23T02:00:41.868+00:...
6	3006	206	6	SAVINGS	4000	2026-03-01T11:50:00.000+00:...	2026-03-23T02:00:41.868+00:...
7	3007	207	7	CURRENT	3500	2026-03-01T11:55:00.000+00:...	2026-03-23T02:00:41.868+00:...
8	3008	208	8	SAVINGS	6200	2026-03-01T12:00:00.000+00:...	2026-03-23T02:00:41.868+00:...
9	3009	209	9	CURRENT	2800	2026-03-01T12:05:00.000+00:...	2026-03-23T02:00:41.868+00:...
10	3010	210	10	SAVINGS	10000	2026-03-01T12:10:00.000+00:...	2026-03-23T02:00:41.868+00:...
11	3012	212	-1	CURRENT	4500	2026-03-01T12:20:00.000+00:...	2026-03-23T02:00:41.868+00:...
12	3013	213	3	UNKNOWN	5000	2026-03-01T12:25:00.000+00:...	2026-03-23T02:00:41.868+00:...

12 rows | 2.21s runtime

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

Gold Layer:

Performing SCD type 1

Source

1 hour ago (1s) 2

```
from pyspark.sql.functions import *
```

1 hour ago (8s) 3

```
df=spark.table("project2.silver.customer")
```

df: pyspark.sql.dataframe.DataFrame = [CustomerID: integer, LastName: string ... 3 more fields]

1 hour ago (11s) 4

```
df.display()
```

(2) Spark Jobs

Table +

	CustomerID	LastName	City	ModifiedDate	ingestion_time
1	209	TAYLOR	Waterloo	2026-03-01T10:55:00.000+00:...	2026-03-24T01:08:45.915+00:...
2	204	SMITH	Ottawa	2026-03-01T10:30:00.000+00:...	2026-03-24T01:08:45.915+00:...
3	207	BROWN	Windsor	2026-03-01T10:45:00.000+00:...	2026-03-24T01:08:45.915+00:...
4	201	PATEL	Toronto	2026-03-01T10:15:00.000+00:...	2026-03-24T01:08:45.915+00:...
5	206	KAPOOR	London	2026-03-01T10:40:00.000+00:...	2026-03-24T01:08:45.915+00:...
6	202	SHAH	Mississauga	2026-03-01T10:20:00.000+00:...	2026-03-24T01:08:45.915+00:...
7	208	WILSON	Kitchener	2026-03-01T10:50:00.000+00:...	2026-03-24T01:08:45.915+00:...
8	205	ALI	Hamilton	2026-03-01T10:35:00.000+00:...	2026-03-24T01:08:45.915+00:...
9	210	MARTIN	Niagara	2026-03-01T11:00:00.000+00:...	2026-03-24T01:08:45.915+00:...
10	203	MEHTA	Brampton	2026-03-01T10:25:00.000+00:...	2026-03-24T01:08:45.915+00:...

10 rows | 10.58s runtime

Importing library and fetching the table using dataframe.
Adding a hash column to the source table:

6	202	SHAH	Mississauga	2026-03-01T10:20:00.000+00:...	2026-03-24T01:08:45.915+00:...
7	208	WILSON	Kitchener	2026-03-01T10:50:00.000+00:...	2026-03-24T01:08:45.915+00:...
8	205	ALI	Hamilton	2026-03-01T10:35:00.000+00:...	2026-03-24T01:08:45.915+00:...
9	210	MARTIN	Niagara	2026-03-01T11:00:00.000+00:...	2026-03-24T01:08:45.915+00:...
10	203	MEHTA	Brampton	2026-03-01T10:25:00.000+00:...	2026-03-24T01:08:45.915+00:...

10 rows | 10.58s runtime

16 hours ago (1s) 5

```
df_hash=df.withColumn("hash",crc32(concat(col("CustomerID"),col("LastName"),col("City"),col("ModifiedDate"),col("ingestion_time"))))
df_hash.display()
```

(1) Spark Jobs

df_hash: pyspark.sql.dataframe.DataFrame = [CustomerID: integer, LastName: string ... 4 more fields]

Table +

	CustomerID	LastName	City	ModifiedDate	ingestion_time	hash
1	209	TAYLOR	Waterloo	2026-03-01T10:55:00.000+00:...	2026-03-24T01:08:45.915+00:...	2239533275
2	204	SMITH	Ottawa	2026-03-01T10:30:00.000+00:...	2026-03-24T01:08:45.915+00:...	1421641846
3	207	BROWN	Windsor	2026-03-01T10:45:00.000+00:...	2026-03-24T01:08:45.915+00:...	216528147
4	201	PATEL	Toronto	2026-03-01T10:15:00.000+00:...	2026-03-24T01:08:45.915+00:...	1081896191
5	206	KAPOOR	London	2026-03-01T10:40:00.000+00:...	2026-03-24T01:08:45.915+00:...	3654961769
6	202	SHAH	Mississauga	2026-03-01T10:20:00.000+00:...	2026-03-24T01:08:45.915+00:...	1161429383
7	208	WILSON	Kitchener	2026-03-01T10:50:00.000+00:...	2026-03-24T01:08:45.915+00:...	2308164953
8	205	ALI	Hamilton	2026-03-01T10:35:00.000+00:...	2026-03-24T01:08:45.915+00:...	1497717255
9	210	MARTIN	Niagara	2026-03-01T11:00:00.000+00:...	2026-03-24T01:08:45.915+00:...	1678911404
10	203	MEHTA	Brampton	2026-03-01T10:25:00.000+00:...	2026-03-24T01:08:45.915+00:...	2956411312

Creating a target table with all the columns and extra four columns. Then importing a deltatable library, assigning the target table to a variable named tablename and creating a delta table object[delta_tgt] using forName function.

SCD-Type1

File Edit View Run Help Python Tabs: ON Last edit was now

Target

16 hours ago (4s) 7

```
%sql
create table if not exists project2.gold.t_customerscd1(
  CustomerID int, LastName string, City string, ModifiedDate timestamp, ingestion_time timestamp,
  createdDate timestamp, createdBy string, updatedDate timestamp, updatedBy string, hash bigint
)
```

OK

15 hours ago (1s) 8

```
from delta import DeltaTable
tablename = "project2.gold.t_customerscd1"
Delta_tgt=DeltaTable.forName(spark, tablename)          #Delta table object creation
```

15 hours ago (<1s) 9

```
display(Delta_tgt)      #Object [Not table]
```

```
<delta.tables.DeltaTable at 0x7f860265fce0>
```

Displaying the delta table object.
Writing the logic to map columns accordingly:

```
12:19 AM (7s)                                                                    10
(Delta_tgt.alias("tgt").merge(df_hash.alias("src"), "tgt.customerid = src.customerid").whenMatchedUpdate(

        condition="tgt.hash != src.hash",
        set = {"tgt.customerid": "src.CustomerID",
               "tgt.lastname": "src.LastName",
              "tgt.city": "src.City",
              "tgt.modifieddate": "src.ModifiedDate",
              "tgt.ingestion_time": "src.ingestion_time",
              "tgt.hash": "src.hash",
              "tgt.updatedDate": current_timestamp(),
              "tgt.updatedBy": lit("databricks_updated")
              }
                 ).whenNotMatchedInsert(
                                                                  values = {"tgt.customerid": "src.CustomerID",
                                                                         "tgt.lastname": "src.LastName",
                                                                         "tgt.city": "src.City",
                                                                         "tgt.modifieddate": "src.ModifiedDate",
                                                                         "tgt.ingestion_time": "src.ingestion_time",
                                                                         "tgt.hash": "src.hash",
                                                                         "tgt.updatedDate": current_timestamp(),
                                                                         "tgt.updatedBy": lit("databricks"),
                                                                         "tgt.createdDate": current_timestamp(),
                                                                         "tgt.createdBy": lit("databricks")})
.execute())
(4) Spark Jobs
DataFrame[num_affected_rows: bigint, num_updated_rows: bigint, num_deleted_rows: bigint, num_inserted_rows: bigint]
```

Resultant target table:

```
12:20 AM (1s)                                                                    11                                                                    SQL
%sql
select * from project2.gold.t_customerscd1
(2) Spark Jobs
_sqldf: pyspark.sql.dataframe.DataFrame = [CustomerID: integer, LastName: string ... 8 more fields]
```

Table +

	ID	LastName	City	ModifiedDate	ingestion_time	createdDate	createdBy	updatedDate	updatedBy	hash
1	209	TAYLOR	Waterloo	2026-03-01T10:55:00.000+00:...	2026-03-24T01:08:45.915+00:...	2026-03-25T04:19:44.001+00:...	databricks	2026-03-25T04:19:44.001+00:...	databricks	223953327
2	204	SMITH	Ottawa	2026-03-01T10:30:00.000+00:...	2026-03-24T01:08:45.915+00:...	2026-03-25T04:19:44.001+00:...	databricks	2026-03-25T04:19:44.001+00:...	databricks	142164184
3	207	BROWN	Windsor	2026-03-01T10:45:00.000+00:...	2026-03-24T01:08:45.915+00:...	2026-03-25T04:19:44.001+00:...	databricks	2026-03-25T04:19:44.001+00:...	databricks	21652814
4	201	PATEL	Toronto	2026-03-01T10:15:00.000+00:...	2026-03-24T01:08:45.915+00:...	2026-03-25T04:19:44.001+00:...	databricks	2026-03-25T04:19:44.001+00:...	databricks	108189619
5	206	KAPOOR	London	2026-03-01T10:40:00.000+00:...	2026-03-24T01:08:45.915+00:...	2026-03-25T04:19:44.001+00:...	databricks	2026-03-25T04:19:44.001+00:...	databricks	365496176
6	202	SHAH	Mississauga	2026-03-01T10:20:00.000+00:...	2026-03-24T01:08:45.915+00:...	2026-03-25T04:19:44.001+00:...	databricks	2026-03-25T04:19:44.001+00:...	databricks	116142938
7	208	WILSON	Kitchener	2026-03-01T10:50:00.000+00:...	2026-03-24T01:08:45.915+00:...	2026-03-25T04:19:44.001+00:...	databricks	2026-03-25T04:19:44.001+00:...	databricks	230816495
8	205	ALI	Hamilton	2026-03-01T10:35:00.000+00:...	2026-03-24T01:08:45.915+00:...	2026-03-25T04:19:44.001+00:...	databricks	2026-03-25T04:19:44.001+00:...	databricks	149771725
9	210	MARTIN	Niagara	2026-03-01T11:00:00.000+00:...	2026-03-24T01:08:45.915+00:...	2026-03-25T04:19:44.001+00:...	databricks	2026-03-25T04:19:44.001+00:...	databricks	167891140
10	203	MEHTA	Brampton	2026-03-01T10:25:00.000+00:...	2026-03-24T01:08:45.915+00:...	2026-03-25T04:19:44.001+00:...	databricks	2026-03-25T04:19:44.001+00:...	databricks	295641131